VERCEL LABS · 开源项目学习笔记

agent-desktop
让 Agent 看见“按钮”，而不只是像素

一份从无障碍树、带快照编号的 ref，到“到底能不能安全重试”的入门指南。重点不是遥控电脑，而是理解它怎样把桌面 GUI 变成 Agent 能读取、能核验的结构化接口。

版本 1.1.0 | JSON envelope 2.6 | Apache-2.0 | macOS / Windows / Linux | CLI + MCP + C FFI

学习基线：官方提交 `6660fbac768dc7e7332837b90ed465684859afa3`
本机实验：仅 version/skills 与固定源码契约核验；未调用桌面适配器。

先纠正宣传里的两个夸张点：它本身没有大模型，不会自己决定下一步；所谓“多个 Agent 同时干活”也不等于多个物理鼠标一起乱点。真正的决策循环在你的 Agent 里，物理输入仍受跨进程交互租约串行保护。

01 它是什么 | 02 为什么用无障碍树 | 03 总体架构
04 核心循环 | 05 snapshot 与 ref | 06 防点错目标
07 headless 与 headed | 08 能否重试 | 09 不完整快照
10 Linux 边界 | 11 MCP 接入 | 12 会话与隐私
13 C FFI | 14 与其他工具比较 | 15 能力与安全实验

01 定位

它到底是什么？是“电脑里的 Agent 大脑”吗？

agent-desktop 是桌面感测与执行工具，不是 Agent 大脑。 它把应用通过操作系统暴露的**无障碍树（accessibility tree）**读成 JSON，并允许 Agent 用元素 ref 调用点击、填值等动作。模型、任务规划、提示词和终止条件都在调用方。



类比：LLM 是驾驶员，agent-desktop 是带语义仪表和受控按钮的驾驶舱；不是自动驾驶策略本身。仓库明确把“嵌入 LLM、做 GUI/TUI、浏览器自动化、宏录制”列为项目范围之外。

依据：README.md；CONTRIBUTING.md；skills/agent-desktop/SKILL.md。

02 原理

为什么不用截图找像素，而要读无障碍树？

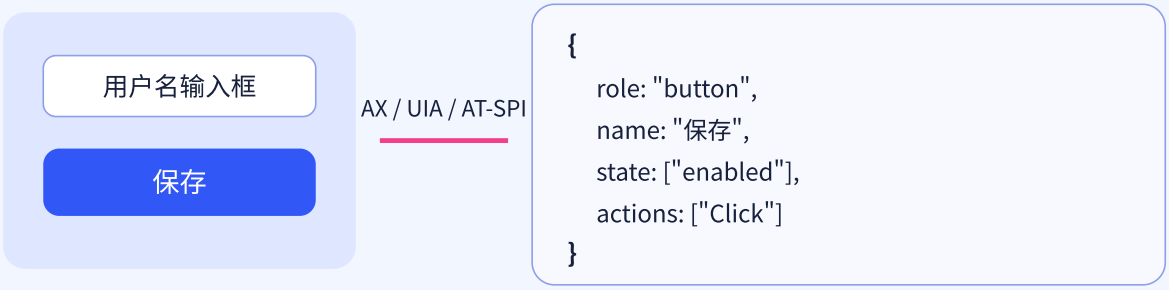
截图回答“哪里有一块像按钮的图”；无障碍树回答“这是一个名为‘保存’的 button，支持 Click，目前 enabled”。后者把**角色、名称、状态和可用动作**直接交给 Agent，省去纯视觉坐标猜测。

像素路线

截图 → 视觉识别 → 估计坐标 → 移动鼠标点击。遮挡、缩放、主题变化都可能让坐标失效。

语义路线

读取 role/name/state/actions → 保存身份证据 → 用 ref 请求语义动作 → 再读结果。

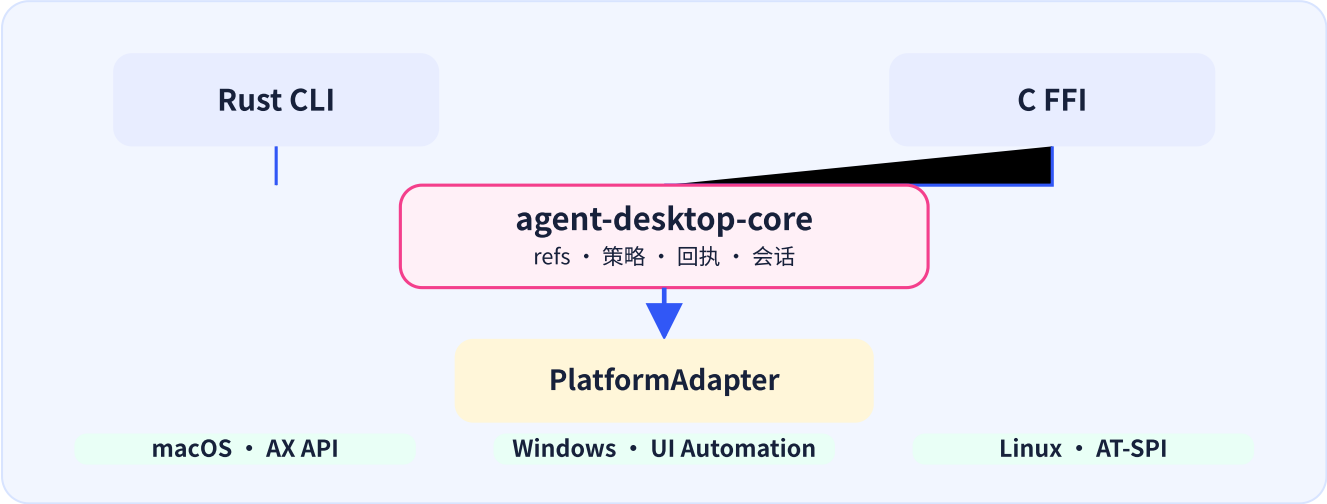


但它不是截图的完全替代品。画布、自定义绘制控件、图表和未暴露语义的界面仍可能需要视觉。截图也能用于人工调试；只是截图和快照都可能含隐私，默认不能进公开日志。

03 架构

CLI、核心库和三种系统接口怎样拼起来？

入口有 CLI 与 C 动态库；二者共用 `agent-desktop-core`。核心通过 `PlatformAdapter` 组合四组能力：Observation、Action、Input、System。各平台 crate 在编译期选择，不是运行时拿一套代码硬套所有系统。



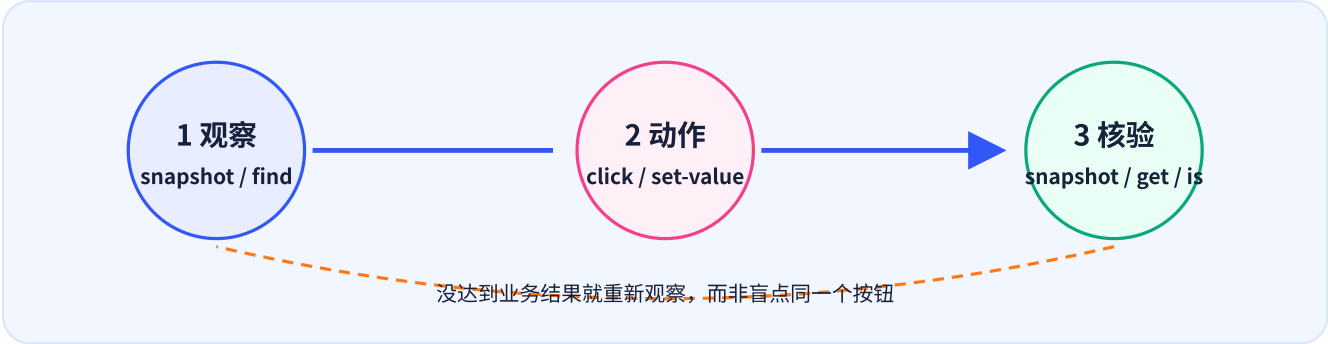
这解释了两个工程事实：核心规则（例如 ref 身份、重试语义）能跨平台复用；真实能力仍由操作系统决定，不能因为命令名相同就承诺每个平台效果相同。

源码：crates/core/src/adapter/mod.rs:30-41；src/main.rs:345-364。

04 工作流

一次可靠操作为什么是“观察 → 动作 → 再观察”？

因为“命令执行完”不等于“业务效果已经发生”。标准循环是先拿到带 ref 的快照，再针对 ref 发动作，最后重新读取界面或业务结果。




```
agent-desktop snapshot --app Finder -i
agent-desktop click @s8f3k2p9:e8
agent-desktop snapshot --app Finder -i
```

这是官方 Quick start 的语义示例，**本次没有在你的电脑上执行**。异步变化用 `wait` 等明确条件，不靠固定 sleep 猜时间。

05

关键术语

snapshot、snapshot ID 和 ref 各是什么?

Snapshot 是某一时刻的无障碍树观察；**snapshot ID** 是这次观察的命名空间；**ref** 是其中一个可操作或可下钻元素的短句柄。当前格式必须是 `@<snapshot_id>:e<N>`。



ref 在单次快照内是确定性的，但**不是跨 UI 变化的永久 ID**。快照还属于会话命名空间：在某个 `--session` 下取得的 ref，后续必须处于同一会话范围。

PowerShell 要把 ref 写成 `'@s8f3k2p9:e8'`，否则 `@` 会触发 splatting 语法。

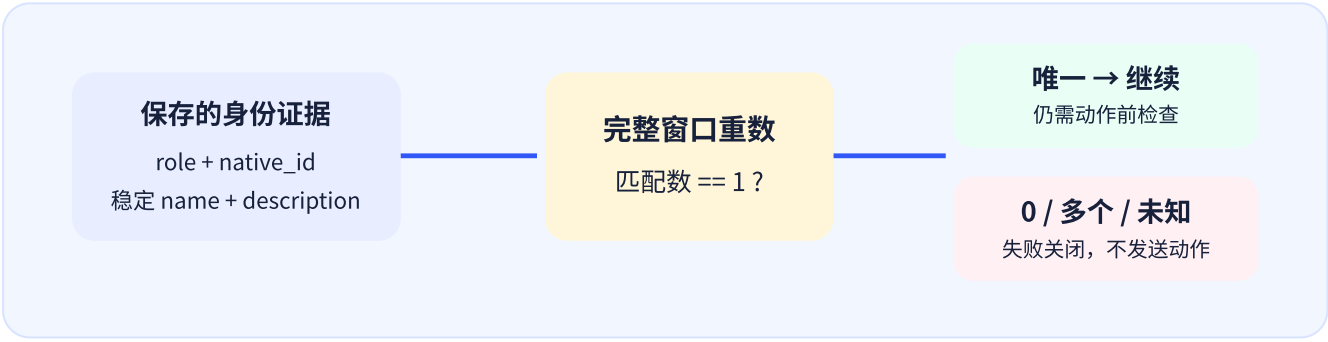
源码：crates/core/src/refs/ref_token.rs；CONCEPTS.md 的 Snapshot / Ref / Session。

06 安全核心

界面变了，它怎样避免点到一个“长得像”的按钮？

严格重识别，证据不足就拒绝，不猜。 在 Windows/Linux（以及没有保留对象地址的 macOS 元素）上，ref 必须在抓取时已由**完整窗口观察**证明唯一；动作前再数一次实时匹配，只有恰好一个才允许继续。

情况	结果	正确动作
元素消失 / 稳定字段变化	`STALE_REF`	重新快照
后来出现相同双胞胎	`AMBIGUOUS_TARGET`	换可唯一识别的控件
抓取时就不是唯一 / 窗口未读全	`identity_not_unique`	提高读取预算或换目标
实时字段没读出来	身份未知，不授权动作	按 TIMEOUT / disposition 处理



坐标、尺寸和可变的输入值不能单独充当身份。`Unknown`（没读到）与 `Absent`（读到了，确认没有）严格分开；否则“读取失败”就会被误当成“没有风险”。

源码：crates/core/src/refs/ref_capture_proof.rs:97-117；ref_identity.rs；docs/json-output.md:146-159。

07

权限边界

headless 是“无界面后台”，还是“绝不碰鼠标”？

这里的 **headless** 指默认的**语义无障碍动作**：不隐式抢焦点、不移动指针、不合成键盘、不借剪贴板。应用窗口仍可显示。**headed** 是显式授权物理输入策略；只有加全局 `--headed` 才进入。

默认 headless

无障碍 API 语义调用

不抢焦点 · 不动鼠标

没有语义路径 → 拒绝

显式 --headed

允许焦点 / 指针前提

具体动作才决定物理路径

不是失败后的自动升级

源码中的默认策略是 `allow_focus_steal: false`、`allow_cursor_move: false`。安全纪律：**headless 失败不能成为你擅自加 `--headed` 的理由**；物理输入需要用户明确许可。

源码：crates/core/src/actions/interaction_policy.rs:9-47；skills/agent-desktop/SKILL.md:71-73。

08 可靠性

失败后为什么不能“再点一次试试”？

因为失败可能发生在动作已经送达之后。重复点击可能重复提交，重复 toggle 甚至会把成功状态又撤回去。因此每个动作结果/错误带 `disposition`：先回答“是否可能送达”，再决定能否重试。

delivery	含义	自动重试
`not_delivered`	明确没有送达	**safe**
`delivery_uncertain`	不确定是否送达	unsafe
`delivered_unverified`	已发送，效果未证实	unsafe
`delivered_verified`	指定后置条件已读回验证	unsafe
`unknown`	没有足够交付证据	不要自动重试



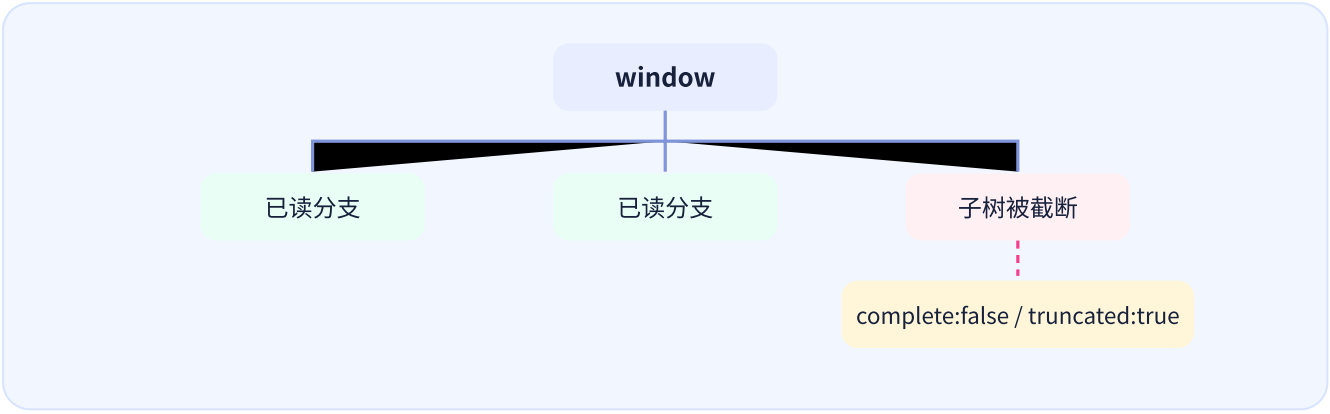
`ok: true` 也可能只是 `delivered_unverified`。可靠调用方应同时看 `error.code`、`disposition` 和独立业务状态。

09

边界情况

树太大、读不完，返回部分结果算成功吗？

完整快照超时后可能返回 `ok: true`，但 `data.complete: false`，并标出截断节点。这是“部分观察成功”，不是“整棵树已读完”。调用方必须主动检查 `complete`。



更重要的区别：`complete` 描述呈现的树；`identity_unproven` 描述 ref 的身份是否得到完整证据。Windows/Linux 上，不能完整读取窗口就意味着其中的 ref 不能被授权使用。

`--root` 下钻会替换原快照内部的 ref，因此不允许建立在不完整观察上，以免误删尚未分配的后代映射。

10 平台差异

“支持 Linux”具体有哪些前提和限制?

官方当前目标是 Ubuntu 24.04+、x86_64/aarch64、glibc 2.35+，并要求目标应用所在图形会话的 AT-SPI 总线、display 和 runtime 环境。普通 SSH/TTY 或无总线容器里“看不到应用”，不等于桌面为空。

能力	X11	Wayland
观察与语义 ref 动作	AT-SPI	AT-SPI
指针、拖拽、滚轮	需显式 headed	应用指针输入拒绝
窗口控制	依 X11 client	原生窗口多受限
截图	依窗口/显示条件	需 portal 同意
剪贴板	支持	需 wl-clipboard

状态目录也是安全边界：Linux 上 `AGENT_DESKTOP_HOME` 若组可写或全局可访问，snapshot 会失败；实验脚本主动设为 `0700`。

同一命令跨平台不代表同一实现。例如 Windows 的 headless `type` 没有 UIA 插入语义路径，应使用 `set-value`；Linux 则可用 AT-SPI EditableText。务必先读 `skills get platform`。

11

AGENT 接入

MCP 模式做了什么？还需要手写工具定义吗？

`agent-desktop mcp` 在 stdio 上提供 MCP 服务。它直接从同一棵 Clap CLI 命令树生成工具名和 JSON Schema，因此 `snapshot` 映射成 `desktop_snapshot`，`session start` 映射成 `desktop_session_start`。

```
{
  "mcpServers": {
    "agent-desktop": { "command": "agent-desktop", "args": ["mcp"] }
  }
}
```



源码限制单条消息 4 MiB、最多 8 个并发 tool call；全局 `--session`、`--headed`、`--trace` 等在服务启动时固定，不能由每次调用临时越权覆盖。

源码：src/mcp/mod.rs:20-32,41-94；src/mcp/catalog.rs:20-73。

12 状态与隐私

会话、trace、截图会不会把桌面内容带出去?

Session 给一组调用提供共享快照命名空间和 trace 目录；它不是云端账号。默认状态根为 `~/.agent-desktop`，可用 `AGENT_DESKTOP_HOME` 改到隔离目录。

事件模式（默认）
每进程追加 JSONL trace 段，合并时按时间、PID、文件内顺序排序。

完整回放（显式开启）
额外保存动作前后 PNG 和 refmap；内容未脱敏，导出 HTML 也可能包含它们。

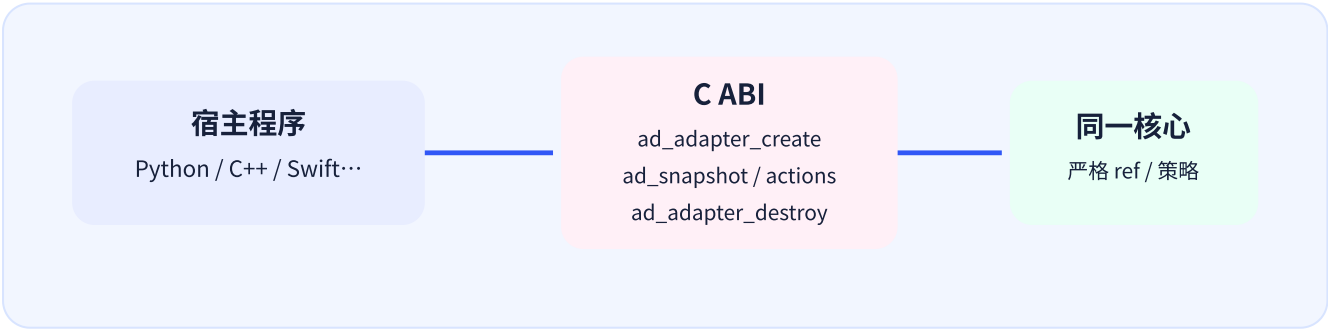


工具本地保存不等于可以公开上传。快照、剪贴板读取、截图、命令输出和 debug artifacts 都可能含私人文件名、文本或账号信息；发布前要单独审阅，文本扫描器看不出截图隐私。

13 进阶

什么时候用 C FFI，而不是每次启动 CLI?

C FFI 把同一适配器以 `libagent_desktop_ffi` 动态库嵌入进程，供 C/C++、Python ctypes、Swift、Go、Node 等调用。适合需要进程内集成、减少进程包装或直接使用 C ABI 的场景；学习规则仍与 CLI 相同。



FFI 有自己的 ABI 主版本、显式资源释放和线程规则；原生元素句柄是线程亲和能力，必须在解析它的线程使用并释放。构建使用 `--profile release-ffi`，因为 FFI 要 `panic=unwind` 才能在边界捕获 panic，普通 release 的 abort 不适用。

源码：crates/ffi/src/lib.rs:3-38；README.md 的 C library。

14 选型 它和浏览器 Agent、传统 RPA、坐标脚本怎么选?

方案	最擅长	主要边界
agent-desktop	有良好无障碍语义的桌面应用；结构化观察和语义动作	平台/应用差异；自定义画布；物理输入仍需授权
浏览器专用 Agent	DOM、网页导航、Web 应用	不覆盖任意原生桌面应用；本项目明确不做浏览器自动化
视觉 / 坐标脚本	没有可访问语义的界面、画布、图像目标	分辨率、遮挡、焦点与身份确认困难
传统 RPA	固定业务流程、录制/编排与企业连接器	流程通常预定义；LLM 自主决策不是天然能力

选择依据不是“更 AI”，而是目标暴露了什么接口

有 DOM
浏览器工具优先

有桌面无障碍语义
agent-desktop 合适

只有像素
视觉路线 / 补应用语义

最佳实践通常是组合，而不是二选一。能走应用 API 就走 API；需要 GUI 时优先语义；视觉只补足确实没有语义的部分。

15 动手实验

不碰真实桌面，怎样亲手验证它的协议与安全边界？

本实验固定 npm `agent-desktop@1.1.0` 与官方源码提交，在本项目 `exercise/` 内局部安装，再跑 3 个无桌面 CLI 命令和 2 组源码契约检查。**harness 白名单只允许 version/skills；没有 snapshot/status、没有 headed、没有输入/剪贴板/窗口动作。**

1 局部安装并隔离 HOME / 状态目录

```
mkdir -p exercise/runtime exercise/home exercise/state exercise/npm-cache
chmod 700 exercise/home exercise/state
HOME="$PWD/exercise/home" npm_config_cache="$PWD/exercise/npm-cache" \
npm install --prefix exercise/runtime --no-audit --no-fund agent-desktop@1.1.0
```

目的：不写全局 PATH，不读真实 `~/.agent-desktop`。本机实际结果：新增 1 个 npm 包。

2 运行无桌面实验脚本

```
python3 exercise/run_safe_lab.py
```

脚本只允许 `version` 与 `skills get`；其他命令直接由 harness 拒绝。环境变量隔离只是纵深防御，`main.rs` 的无适配器分支才是命令路径证据。

> ✅ 实测：`{"ok": true, "passed": 5, "desktop_commands": "version_and_skills_only"}`

3 确认真实二进制与协议版本

```
agent-desktop version
```

> ✅ 实测：`data.version = 1.1.0`，`data.os = linux`，envelope `2.6`，退出码 0。

4 读取二进制内嵌的通用与平台技能

```
agent-desktop skills get desktop
agent-desktop skills get platform
```

> ✅ 实测：通用文档 6,997 字符；自动选择 `agent-desktop-linux`，17,209 字符。技能随二进制版本分发，减少文档漂移。

5

静态核对严格 ref 与权限探测顺序

```
crates/core/src/refs/ref_token.rs
src/main.rs
crates/linux/src/system/permissions/mod.rs
```

脚本断言 `@` 前缀、`:e` 分隔、快照 ID 和本地 ref 校验都在固定源码中；同时确认权限探测先于 ref preflight。

> ✅ 源码契约通过；不把 `click @e1` 当无桌面实验，因为 Linux 可能先连接 AT-SPI 并读取注册表。

6

静态核对交互与重试策略

```
crates/core/src/actions/interaction_policy.rs
crates/core/src/actions/delivery_semantics.rs
```

> ✅ 源码契约通过：headless 默认禁止抢焦点/移动指针；只有 `NotDelivered` 映射安全重试，Unknown 不会被当成安全。

完整机器可读证据：`exercise/results/safe-lab.json`；复现说明：`exercise/README.md`。

没有验证的部分：真实 AT-SPI 读取、权限弹窗、真实点击/输入、MCP 长驻调用、C FFI、macOS/Windows 行为和 Rust 源码编译。本机没有 Rust 1.89，本指南不虚报这些测试。

能力清单

学完这个项目，我能用它做什么？

1. 桌面应用语义读取器
把按钮、菜单、输入框、状态变成 Agent 可处理的 JSON。

2. 自有桌面应用的自动化验收
用观察→动作→再观察验证明确状态，不只截一张“看起来成功”的图。

3. 后台表单与设置助手
在有无障碍语义路径时减少抢焦点和鼠标干扰；不支持时明确拒绝。

4. 给 Agent 加统一桌面工具面
CLI 可脚本化，MCP 可直接接宿主，C FFI 可嵌入进程。

5. 多 Agent 分工的受控桌面会话
共享快照命名空间与 trace；物理输入仍按租约串行，不是无约束并发。

6. 应用可访问性诊断
从树中看出控件是否缺名称、角色或动作，反向改进自有应用的可访问性。

最重要的收获不是 60 个命令，而是四条规则：目标必须重新识别；权限不能隐式升级；重试取决于是否送达；业务成功必须独立核验。

证据与边界

本指南的依据

- 官方仓库：github.com/vercel-labs/agent-desktop，固定提交 `6660fbac768dc7e7332837b90ed465684859afa3`。
- 官方 npm 包 `agent-desktop@1.1.0`，Node 要求 18+，许可 Apache-2.0。
- 源码重点：CLI 分发、PlatformAdapter、ref token/capture proof、InteractionPolicy、MCP 目录、FFI。
- 实验只执行 version/skills 与静态源码断言；初版 ref 运行用例因权限探测顺序被独立审核否决，已移除。所有数字来自 `exercise/results/safe-lab.json`。

上游 README 的示例命令只用于解释，不在本指南中冒充本机桌面实测。平台能力会随 OS、应用和后续版本变化，实际使用前请运行对应版本的 status 与 skills get platform，并在用户授权的隔离测试应用中验证。